

Financial Advisor Bot

Explainable decision support for beginner investors

CM3070 Computer Science Final Project

University of London

September 2026

Student number MS1329

Artificial Intelligence

4.2 Project Idea 2 Financial Advisor Bot

Code repository github.com/sloka-bot/financial_advisor_bot

Implementation and evaluation report

An educational application integrating price and news analysis, constrained portfolio proposals and explanations. The evaluation finds limited predictive improvement and identifies the conditions under which the software and its outputs can be assessed.

Contents and word counts

1 Introduction	3
2 Literature review	5
3 Design	10
4 Implementation	15
5 Evaluation	22
6 Conclusion	33
References	35

Word counts cover body prose, including code examples. Tables, headings, captions, references and front matter are excluded. Counts use whitespace-separated words.

Introduction: 850 words

Literature review: 2,380 words

Design: 1,679 words

Implementation: 2,459 words

Evaluation: 2,291 words

Conclusion: 836 words

Total counted text: 10,495 words

Maximum: 10,500 words

1 Introduction

Chapter word count 850

1 1 Project context and motivation

The Financial Advisor Bot is a web application for beginner investors. It combines stock analysis, portfolio suggestions and explanations in one place. A user can enter a budget, inspect a proposed allocation, ask questions and compare a stock forecast with historical outcomes. Recommendations require review before they change the saved portfolio.

I chose this project because I am interested in algorithmic trading and plan to pursue a master's degree in finance. When I began trading, I found it difficult to decide which information mattered and how to interpret conflicting signals. A positive headline could appear alongside falling prices, while indicators often suggested different actions. I wanted to build an application that explained its calculations and made those disagreements visible.

This project follows the CM3070 Artificial Intelligence template, section 4.2, Project Idea 2: Financial Advisor Bot. The template combines financial analysis, an investment strategy and an interface that explains recommendations to nontechnical users. I developed separate research and application workflows so that a user does not need to download a dataset or train a model before asking a question.

The presentation of information matters alongside its accuracy. Attention-driven buying has been associated with poor subsequent returns among retail traders [1]. A prominent stock or confident-looking score can encourage action without explaining the risk. The application therefore separates a forecast from a recommendation and shows the proposed transaction before asking for approval.

1 2 Aim and objectives

The aim is to build and evaluate an explainable financial advisor that combines historical prices and financial news. The project examines whether these inputs improve forecasts, whether portfolio suggestions satisfy financial constraints, and how the calculations can be presented clearly.

The objectives are to prepare dated price and news data; compare technical, sentiment and combined forecasting models; construct portfolios within budget and risk limits; explain model outputs; and support new portfolios and imported holdings. Historical Standard & Poor's 500 (S&P 500) membership defines the experimental universe. I chose this manageable subset to shorten training time and test whether the workflow operated consistently across stocks. It is a computational scope choice, not a requirement of the forecasting algorithms. Brokerage execution, intraday trading and a complete assessment of personal financial circumstances remain outside scope.

Three quantitative questions guide the experiments. Do learned models improve on simple baselines? Does news sentiment add information to price-derived indicators on the same observations? How do portfolio allocation methods compare after trading costs? Interface development addresses a separate question: how should these outputs be arranged so that a beginner can find and review a recommendation?

The project defines success in two separate ways. Software success means the intended calculation completes with valid inputs and stays understandable when evidence is missing. Research success means a defensible answer to the comparison questions, even when that answer is negative. I did not make beating a market index a condition for presenting results, because that

would reward picking whichever configuration happened to look strongest. The evaluation therefore reports the comparisons that failed alongside the application features that work.

1 3 Project components

Extreme Gradient Boosting (XGBoost) processes numerical features. A Long Short-Term Memory (LSTM) network uses sequences of past observations to estimate future returns. Financial Bidirectional Encoder Representations from Transformers (FinBERT) classifies financial news. Optional research implementations include a Gated Recurrent Unit (GRU) and a Hidden Markov Model (HMM) selector. Proximal Policy Optimisation (PPO) is evaluated for portfolio allocation. The final evidence set does not contain the optional GRU or HMM results.

SHapley Additive exPlanations (SHAP) show how individual inputs affect the classifier's output. Ollama provides a local language-model connection for readable explanations. The HeyGen LiveAvatar integration adds spoken replies through an on-screen avatar. It is intended as a disability-friendly alternative for people who find typing or sustained reading difficult, and for users who feel more comfortable speaking than using text chat. Written replies remain available so that users can read the numbers directly.

The main technical work is joining these components correctly. Price fields have different purposes, news must be available before it affects a decision, and a future return cannot be used as a training label before the outcome is known. Portfolio suggestions also need to remain affordable after rounding shares and deducting fees. Individually these are demanding methods, a transformer language model, a recurrent sequence network, a reinforcement-learning allocator, a Gaussian hidden-Markov regime selector and isotonic probability calibration, so the harder contribution is making them share one leakage-controlled pipeline rather than run in isolation.

1 4 Evidence and report organisation

The report uses saved experimental results and software checks completed during the final review. The forecasting results are close to simple baselines. The portfolio experiments contain large differences between methods, but their short duration and unequal trading rules limit direct comparison. These findings are examined in Chapter 5 rather than treated as evidence that a complex model is necessarily better.

Earlier feedback identified excessive information in the initial interface. The revised application uses five views: Dashboard, Portfolio, Stock Predictor, Chatbot and Tools. This is a documented design response to feedback; there are no recorded task-completion measurements for the revised interface. LiveAvatar and Ollama have implemented connections, but successful live provider sessions are not established by the available evidence.

2 Literature review

Chapter word count 2,380

2.1 Financial prediction and simple comparisons

Financial forecasting asks whether information available today helps estimate a later outcome, and the choice of outcome matters: predicting the direction of a move is a different problem from estimating the size of a return, and neither directly determines how much to invest. This project therefore evaluates direction, return error and portfolio performance as separate questions.

XGBoost adds regularisation and efficient handling of structured data to gradient boosting [2], which builds a prediction by adding trees that each reduce the current model's errors. For a table of momentum, volume and news features this is attractive because trees represent interactions, such as a momentum signal that only helps when volatility is low, without the separate term a linear model would need. That same flexibility is its main risk: a tree ensemble can fit patterns that do not persist beyond training, so its worth must be judged against a linear baseline on the same data, not in absolute terms.

Machine learning has been shown to add value in empirical asset pricing, especially for nonlinear relationships [3]. This motivates the approach but does not transfer directly, because the predictors, return horizon and portfolio construction here differ, so a published return is not a benchmark this application can fairly be scored against. The defensible test is the improvement over simpler methods on the same observations.

Three baselines make that test concrete. Logistic regression is a linear reference for direction, an always-up classifier measures what class imbalance alone achieves, and a historical-mean forecast asks whether a learned return model beats a single fixed value estimated in development. All three are cheap and expose a recurring pitfall: a model can look accurate simply because upward outcomes are more common.

Technical indicators summarise past prices and volumes, moving averages for position and trend, momentum for recent change, volatility for variation, but many share inputs and are strongly correlated, so adding features can raise complexity without adding information. An ablation, which compares the full model against the same model trained without a feature group, tests each group's actual contribution.

The training loss decides which errors matter. Squared error penalises a few large mistakes more than absolute error, while directional classification treats any small positive return as an upward outcome regardless of whether trading it would cover costs. None of these objectives measures the user's final wealth directly. Reporting several metrics exposes the differences but also creates room to select a favourable one, so the primary outcome should be fixed before the test sample is examined.

A return target also makes stocks with different prices comparable: a one-dollar error means little for a thousand-dollar stock and a great deal for a ten-dollar one. The application converts the estimated return back to a price for display, which aids familiarity for beginners but does not make the estimate itself more accurate.

2.2 Learning from sequences

The LSTM maintains a memory state and uses gates to control what is retained, replaced and exposed to the next layer [4], letting an input window hold a sequence of observations rather than a single day's summary. Reported applications to financial market prediction motivate this

architecture [5], but performance depends heavily on the market period, trading rule and data preparation, so it is investigated here rather than assumed superior.

The project uses a thirty-session LSTM window, which can represent patterns over time but forces choices about window length, feature scaling and incomplete histories. Longer windows cost more memory and may carry stale relationships; shorter ones are cheaper but may omit relevant change. The thirty-session length is a fixed design assumption, not a tuned optimum, since no window-length search was run, so it is reported as a modelling choice whose sensitivity is untested.

The experimental GRU is a small comparison model with a single input time step. Although both are recurrent networks, this GRU does not test the same temporal information as the deployed thirty-session LSTM: a strong GRU result would suggest the current observation holds a useful nonlinear pattern, not that a long sequence helps. No numerical result is reported because the final experiment record excludes this optional run.

Computation is a further difference. The recorded LSTM run contains 899,153 sequences; materialising thirty observations of twenty-four 32-bit values for each would need about 2.59 billion bytes for inputs alone. Indexed windows and batched prediction avoid holding every inference window in memory, at the cost of extra data-loading work that matters on a personal computer.

The recurrent design also adds correlated structure: consecutive windows overlap heavily, so thousands repeat much of the same information, and a network can fit these repeats without learning a relationship that survives a new period. A random split would worsen this by placing near-identical histories on both sides, so chronological evaluation is required even when the sequence count looks reassuring. Training adjusts the network weights through backpropagation through time, passing prediction errors through the unrolled sequence. Gates provide a route for information and gradients to persist, but they do not guarantee that a useful financial relationship exists. Dropout and validation-based stopping reduce some fitting risk; neither creates independent observations or repairs a poorly defined target.

A linear model on several lagged returns would have isolated the added value of recurrent memory more cleanly than a comparison with a constant, but it was not included in the final set. The LSTM comparison can therefore show whether the network beats the recorded baselines, but not separate the benefit of its sequence inputs from that of nonlinear modelling.

2 3 Financial news and sentiment

BERT learns word representations from surrounding context [6], and FinBERT adapts this to financial language [7]; the pretrained model used here returns positive, negative and neutral classes [8]. Context matters: unlike counting positive words, the model can separate lower costs from lower revenue.

The output describes tone only. It does not say whether news is new to the market, how far the price has already moved, or whether an event affects the chosen horizon, so a correct positive label can still precede a negative return. Sentiment therefore enters the forecaster as an input, not as a direct buy instruction.

The FNSPID dataset combines financial news with price records [9]. It supplies the historical experiments, though its published size exceeds the subset usable here after local filtering removes unsuitable dates, duplicates and missing text; usable coverage is then measured in stock sessions with news, alongside the article count.

Publication time governs usability: a story released after a close cannot explain a decision made at that close. Dates without times create uncertainty around weekends and early closures, and

assigning them to a later eligible session is conservative but can miss information available earlier that day, a coverage loss to keep in mind when reading a weak sentiment result.

Missing news is not neutral news: a stock with no article has no new textual evidence, whereas a neutral article is an observed classification. The model therefore receives article counts and a no-news flag alongside scores, and a retained score can describe recent sentiment while the count stays zero on a session with no fresh article.

Aggregation is another choice. A daily mean gives a brief note the same weight as a major earnings release; confidence weighting rescales by the classifier's certainty, not the event's economic importance; and repeated coverage of one event can dominate a day even after exact duplicates are removed. These effects motivate keeping article counts and disagreement rather than trusting one average.

Using a pretrained model cuts training cost but adds uncertainty, because its financial vocabulary may differ from the collected articles. Without a manually labelled sample, poor forecasting cannot be blamed specifically on misclassified sentiment; the unanswered prior question is whether the labels are correct before asking whether they predict returns. This also separates two forms of learning in the project. FinBERT supplies representations learned outside this dataset, whereas the return models fit relationships in the project's development period. The local language model generates wording from evidence; its fluency is not a further validation of either model's prediction.

2 4 Portfolio allocation

Mean-variance portfolio selection balances expected return against return variance and the covariance that captures how assets move together [10]. Two individually volatile assets can combine into a lower-variance portfolio when they do not move together. Allocation therefore requires more than selecting the largest forecasts.

These estimates are fragile: small errors in expected returns can swing optimised weights sharply, especially without position limits [11]. Concentration caps, cash floors and a risk penalty restrain the allocation here but make it depend on explicit policy settings; the conservative, moderate and aggressive labels describe those settings, not a full assessment of a user's needs.

PPO learns a policy by interacting with a simulated environment, its clipped objective limiting how far the policy shifts per update [12]. For allocation the environment fixes the observations, allowed weights, costs and reward, and a policy can exploit flaws in those rules, so checking the simulator matters as much as the learning algorithm.

Comparing against a fixed allocator is hard when trading frequency differs: daily rebalancing creates more opportunities and costs than rebalancing every twenty-one sessions, and risk statistics shift with the measurement interval. The report therefore states endpoint returns beside the differing rules rather than ranking them by Sharpe ratio.

Markowitz and PPO answer different questions: Markowitz picks weights from explicit estimates at one point, while PPO learns a rule across simulated transitions whose reward defines success. A rising training curve only shows success under that reward in that environment, so test wealth is still needed, and changing the reward, window or asset set effectively changes the problem. PPO uses an actor network to select actions and a critic to estimate expected future reward. The advantage compares the observed outcome with that expectation, directing updates towards actions that performed better than anticipated. Clipping restricts the probability change during optimisation rather than placing a financial loss limit on the portfolio. Cash floors, costs and feasible weights must still be enforced by the environment.

The number of independent market histories matters more than the seed count. Several seeds reveal sensitivity to initialisation within one history but do not create independent recessions or recoveries; an allocator that survives seeds on a short rising period can still fail when prices fall together or costs rise. The three-seed comparison addresses initialisation while leaving those conditions largely untested.

2 5 Explanations and probability quality

SHAP assigns each feature a contribution relative to a reference output [13]: a local waterfall explains one prediction, a global plot summarises contributions over a sample. Both help identify influential inputs, though correlated indicators complicate reading because related features can share or redistribute contributions.

Ranking and probability accuracy are distinct. Calibration can adjust scores so predicted probabilities better match observed frequencies [14]; a model may rank upward outcomes correctly yet still output probabilities that are too high, or emit a near-constant probability that merely echoes the class rate without identifying which stocks rise.

The application calibrates scores and displays explanatory evidence separately, and the language layer receives the numerical result and context rather than making a new forecast, with a fixed template as fallback if the service fails. This keeps the calculation usable, though the generated prose's clarity still needs reader assessment.

A feature explanation is not a causal one. A large SHAP value means the fitted model relies on that feature for the chosen output and reference, not that changing the indicator would move the price, an important caveat for correlated measures from the same price history. The beginner explanation should describe the model's response without presenting an association as a causal mechanism.

2 6 Reliable evaluation

Leakage arises when a model receives information unavailable at prediction time, whether from future labels, preprocessing fitted on later data, or selecting stocks because their future histories are complete [15]. A chronological split stops random past-future mixing but not the other two, so data selection and preprocessing need their own checks.

Repeatedly inspecting a test period also changes its role: a configuration chosen after seeing a result is partly a response to it, and backtest overfitting grows with the number of strategies tried [16]. The report therefore separates the recorded comparisons from later exploratory readings, including the apparent gain from removing a feature group.

Together these methods form an educational application whose research contribution is comparing technical and sentiment inputs under explicit timing rules, supported by explanations and portfolio checks. It introduces no new learning algorithm, so the evaluation must show whether the extra components improve the relevant outcome enough to justify their complexity.

2 7 Comparison with existing platforms

The earlier design compared Wealthfront, Bloomberg Terminal and TradingView. Adding Portfolio Visualizer separates investment management, professional analysis, charting and research simulation. Their official documentation is credible evidence of advertised functions, but it is commercial material, not independent evidence of improved returns or beginner understanding.

Platform	Focus	Key function	Relation to this project
Wealthfront [17]	Robo-advice	Risk questionnaire, diversified allocation, auto-rebalancing	Adopts risk-then-allocate; narrower universe
Bloomberg PORT [18]	Professional analytics	Positions, risk and performance in one view	Beginner scope, not professional depth
TradingView [19]	Charting, paper trading	Chart analysis with simulated positions	Separates forecast from executed position
Portfolio Visualizer [20]	Backtesting	Compare and backtest allocations	Shows the need for a fixed benchmark
This project	Beginner decision support	Constrained proposals, explanations, spoken avatar, approval step	Adds a review step between automation and analysis

Table 1 Comparison of existing platforms and this project. Platform functions come from official documentation, which is commercial material rather than independent evidence of returns or beginner understanding.

Wealthfront makes risk assessment part of the investment journey rather than an isolated setting [17]. My application adopts that sequence, but its three risk tiers and equity universe are narrower. Offering individual-stock forecasts does not establish an advantage over diversified investing, particularly when those forecasts barely improve on a constant baseline.

Bloomberg PORT is a useful reference for linking holdings to analysis rather than presenting disconnected prediction scores [18]. Its professional scope differs from this project's beginner focus. I therefore separate the overview from detailed analysis; I cannot conclude that professional users find the Terminal difficult or that it lacks explanations without studying those users and features.

TradingView's paper trading challenges the earlier draft's description of it as merely displaying indicators [19], and motivates a visible distinction between a forecast and an executed position. My contribution is a constrained review journey with shared text and avatar replies, not a claim to replace its wider analytical tools.

Portfolio Visualizer makes comparison between allocations a central activity [20], while this application adds conversational explanations around saved holdings. The research lesson is that an attractive return is insufficient without a specified benchmark and common assumptions. The missing fixed benchmark in my results remains a weakness even though a portfolio interface works.

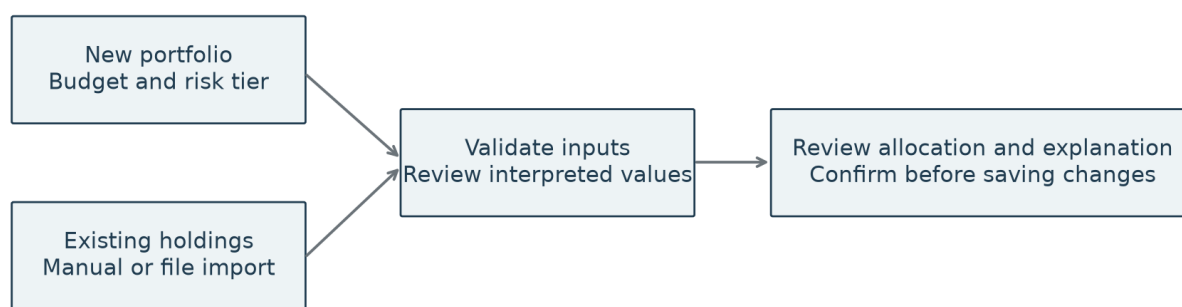
Across these platforms, automation reduces manual work while analysis tools expose more control. This project places the review step between them: calculations generate candidates, users approve changes, and explanations accompany the decision. That is a concrete design contribution. Its educational value still needs observation of beginner tasks; listing more algorithms than a competitor would not demonstrate that value.

3 Design

Chapter word count 1,679

3.1 User journeys and requirements

The application supports two starting points. A new investor enters a budget and risk preference, then reviews a proposed allocation. An existing investor enters holdings manually or imports a spreadsheet, checks the imported values and reviews proposed changes. Both journeys end with an approval step before the saved portfolio changes.



A preview does not execute a trade or change saved holdings.

Figure 1 The two user journeys converge on input validation and a review step. Schematic workflow, not participant-study evidence.

The main requirements are valid financial inputs, affordable allocations, explanations connected to the displayed result and a clear distinction between holdings and proposals. The user should be able to inspect a forecast without training a model. Missing data should have an understandable state, and failures in an optional service should not prevent access to the rest of the application.

The initial interface was described in the supplied feedback as overwhelming because it showed too much information at once. I reorganised it into five views. Dashboard presents money and holdings first. Portfolio contains proposals and approval controls. Stock Predictor separates a current estimate from historical comparisons. Chatbot holds text and avatar conversation. Tools contains the additional calculators. Detailed indicators are available when needed instead of occupying the first screen.

I prioritised reviewability over reducing every journey to one click. Import preview adds a step, but allows a user to identify an incorrect quantity before it affects the portfolio. Proposal approval similarly separates advice from account state. These choices reflect the cost of an unnoticed financial input error. A decorative improvement cannot compensate for a screen that makes an unapproved suggestion appear to be an existing holding.

The interface also needs consistent units. Cash and invested value use currency, allocation uses percentages, and the forecast horizon uses trading sessions. Displaying those units close to a number reduces the need to remember conventions from another page. Colour supports the distinction between gains and losses, but the sign and label must carry the same information for users who do not distinguish the colours.

3 2 System structure

The browser handles forms, navigation and presentation. The Python backend validates requests, loads saved data and calls the appropriate analytical module. An application programming interface (API) connects them through routes for portfolios, predictions, imports, chat and evaluation. FastAPI supplies the request-routing and validation framework.

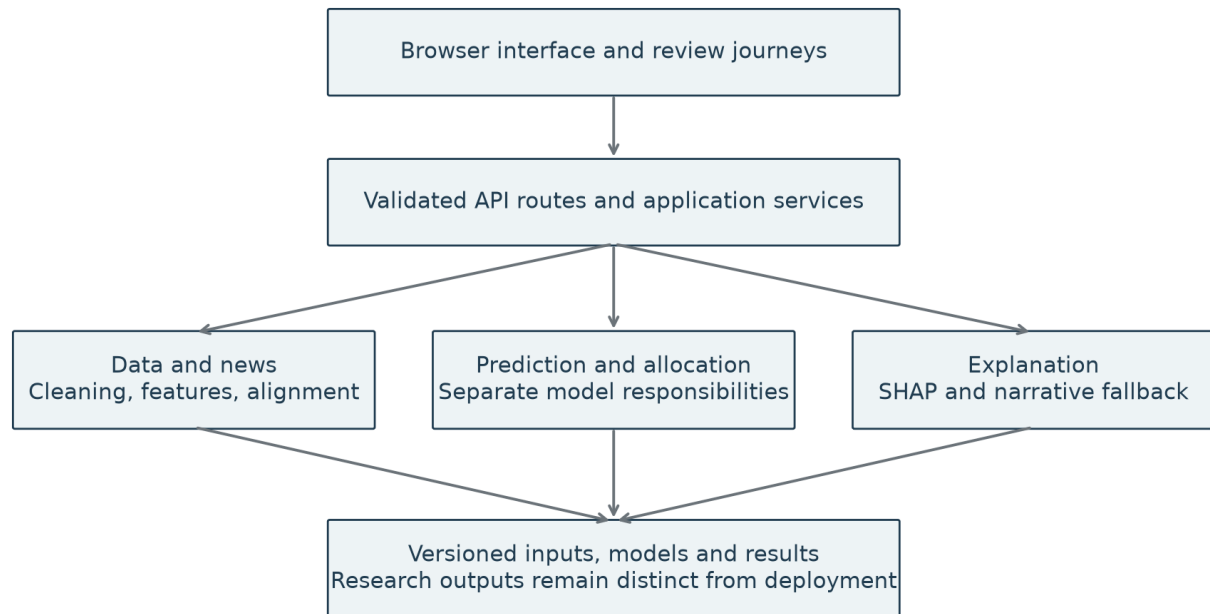


Figure 2 Component responsibilities. Research artifacts and interactive advice share calculations but retain separate evaluation identities.

Training runs separately from ordinary application use. Research scripts prepare datasets and produce experiment files; the application loads saved models for prediction. Shared modules provide the price definitions, financial calculations and explanation inputs used by both paths. Model metadata records the feature order so that a saved model is not passed a different arrangement of columns.

The backend separates routes from the training service and analytical code. A spreadsheet error can then be handled in the import route without changing model training. Similarly, the share-allocation calculation can be checked independently of the browser. The frontend uses separate page modules and a shared request helper, with the avatar connection contained in its own component.

3 3 Price and news alignment

The data pipeline keeps adjusted analytical prices and unadjusted execution prices. Adjusted history supports return calculations across corporate actions. The unadjusted price is needed when converting a cash amount into shares. These fields are not interchangeable: a historical price altered for a later stock split may not equal the amount paid for one share on that date.

Price records are sorted by date and duplicates removed before targets are calculated. Short gaps can be filled from earlier observations, while suspicious jumps receive quality flags. Filling from a later observation would introduce future information. An indicator also needs enough previous observations for its window; a two-hundred-session average cannot be considered complete after only twenty sessions.

News is mapped to the first eligible market close after publication, using an exchange calendar that includes holidays and early closes. A date-only article is handled conservatively because its time of availability is unknown. The fusion stage joins the resulting daily sentiment fields to each stock's price-derived features.

A label is the outcome the model is trained to predict. Here it is a later adjusted-price return over one, five or twenty-one trading sessions. Its end date is stored with the row. A training row must be removed when its outcome extends into the next evaluation period. The newest rows remain useful for live prediction even though their future outcomes are not yet known.

The data flow can be understood as four times: publication, observation, decision and execution. Publication belongs to the news source. Observation is the latest price or article available to the calculation. The decision uses only that information, while the simulated trade occurs at the following close. The return used to judge that trade begins after execution. Keeping these times separate prevents a strategy from benefiting from a movement that happened before it could have held the position.

For example, an indicator computed using Tuesday's closing price cannot support a trade made earlier on Tuesday at that same close without an additional execution assumption. The implemented comparison delays the fill to the next observed close. This sacrifices some possible return but gives an explicit, reproducible timing rule. Missing execution prices remain missing evidence; replacing them with an average would change both affordability and the simulated return.

3 4 Forecasting and allocation

The evaluated research comparisons use fixed technical and sentiment rules and learned return models. Optional modules provide an equal-weight combination of predictions and a market-state selector, which are not represented in the final result file. The application's classifier estimates upward direction, while its LSTM estimates return from a sequence. These outputs have different meanings and are labelled separately in the interface.

The portfolio optimiser uses estimates of return and covariance with profile-dependent constraints. The historical portfolio experiment uses price-history inputs; it does not pass the evaluated stock forecasts into PPO or Markowitz. Its results therefore assess allocation methods under those inputs. The current scope is narrower than a complete forecast-driven trading policy.

Budget and risk tier directly affect the user's allocation. Other collected preferences provide context, but the report does not treat them as optimised inputs unless they change the calculation. Table 2 gives the implemented limits. Risk aversion controls the strength of the penalty on portfolio variance: a larger value makes variation more costly within the objective.

Risk tier	Maximum asset weight	Minimum cash	Risk aversion
Conservative	15%	20%	8.0
Moderate	25%	5%	3.0
Aggressive	40%	0%	1.0

Table 2 Configured portfolio constraints. These are design settings, not empirically validated suitability categories.

3 5 Share quantities and cash

A weight is the fraction of the budget allocated to a stock. To find an initial share quantity, multiply that fraction by the budget, divide by the execution price and round down. For example, a 20% allocation from a \$5,000 budget assigns \$1,000 to a stock. At \$120 per share, the initial quantity is eight shares, costing \$960 before fees.

```
allocated_cash = budget * target_weight
initial_shares = floor(allocated_cash / execution_price)
```

The example explains the arithmetic; the final allocation also checks cash, fees and concentration limits. Rounding can change the proposed proportions, particularly with a small budget or expensive stock. Any leftover amount stays as cash. The required accounting identity is that invested value, transaction fees and remaining cash add up to the starting budget.

For an existing portfolio, target holdings are compared with current holdings: a reduction proposes a sale and an increase proposes a purchase. Previewing records no transaction. Buy more accepts a whole-share quantity, while Sell closes the selected holding. Both use execution prices, deduct fees and update cash. Build proposes up to five additions funded by available cash, with limits checked against the combined portfolio rather than the new purchases alone.

3 6 Explanations and voice interaction

SHAP provides feature contributions, while the language layer turns structured results into sentences. A forecast probability, an expected return and a combined recommendation score are presented as different quantities. A score built from fixed rules must not be labelled as a statistically calibrated probability.

The HeyGen avatar is intended to make conversation more accessible to users who find speaking easier than typing, including some users with motor or reading difficulties. The avatar receives the same answer shown in chat. Microphone transcripts return through the same chat route, so choosing voice does not invoke a separate financial advisor. Text stays visible for users who prefer reading, cannot hear the audio or want to check an amount.

The user controls the microphone and can end the session. Leaving Chatbot stops the avatar connection. If the video service is unavailable, text chat remains available. These are accessibility-oriented design choices; the project has not measured their benefit for disabled users.

Reliability requires preserving meaning when a component is unavailable. No news, a zero sentiment score and a failed scoring job represent different states. Likewise, zero investment growth differs from an account with no recorded history. The design retains these distinctions rather than filling every empty field with a plausible value. A short explanation of unavailable evidence is preferable to a complete-looking display that silently changes the calculation.

External conversation services are optional dependencies. Their session credentials should remain separate from saved financial data, and microphone access should end when conversation ends. Sending the already constructed answer to the avatar limits disagreement between spoken and written figures. It does not remove transcription errors or establish that speech is easier for every user; the written route remains necessary.

3 7 Evaluation design

Forecasting uses chronological development and test periods. Preprocessing is fitted on earlier data, and training labels that overlap a later partition are excluded. Comparisons use the same stock-date rows wherever possible. The LSTM requires an additional history check, so its baselines are rescored on its eligible sample.

Two further controls protect the split. An embargo of twenty-one sessions, the longest label horizon, separates the development and test blocks, so a development row's forward-looking label cannot overlap the first test dates and leak their outcomes into training. Stock eligibility is also resolved point-in-time: each date uses the index constituents as of that date, so a company contributes rows only while it belonged to the index. This guards against the survivorship bias of scoring only firms that lasted to the present, although delisted names with incomplete downloaded history can still be under-represented.

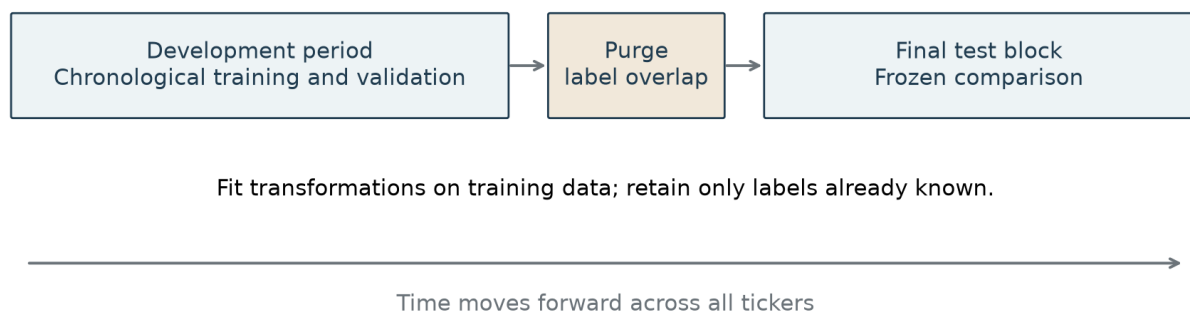


Figure 3 Chronological evaluation and label maturity. Block widths are schematic, not dataset proportions.

The tests cover input validation, date handling, missing values and financial accounting. Historical experiments measure prediction and simulated returns. Interface review checks the navigation and the separation of forecasts, proposals and holdings. The response to the earlier interface feedback is documented through the changed layout and application screenshots. No formal user-evaluation protocol or participant results are included.

4 Implementation

Chapter word count 2,459

4 1 Application modules

Python performs data preparation, prediction and portfolio calculations. FastAPI exposes these functions to the browser, which uses JavaScript for interaction. Routes are grouped under backend/api/routers, while the training workflow is held in a service module. Frontend pages have separate modules for the five navigation views.

Requests are checked before expensive calculations begin. A negative budget or an invalid date range is rejected with a recoverable response. Numerical outputs are converted into valid JavaScript Object Notation (JSON), the format used for API responses and saved results. The browser distinguishes a valid zero from an unavailable value; a missing return is not replaced with an invented average.

4 2 Price features and labels

Feature generation uses each stock's ordered price history. The experimental technical set contains thirty-six predictors: twelve trend, twelve momentum, six volatility, three volume and three strength measures. Eight sentiment fields bring the combined experimental set to forty-four when all fields are present. A master file contains additional identifiers, prices and target columns, so its total column count is not the feature count supplied to a model.

The future return is calculated as the later adjusted close divided by the current adjusted close, minus one. A value of 0.02 represents a 2% increase. The direction label is one for a positive return and zero otherwise, but only where the future return is known. The following extract preserves unknown outcomes as missing values.

```
h = PREDICTION_HORIZON
future_return = df["close"].shift(-h) / df["close"] - 1.0
df["target_return"] = future_return
df["target_direction"] = np.where(
    future_return.notna(), (future_return > 0).astype(float), np.nan
)
df["target_end_date"] = pd.Series(
    df.index, index=df.index
).shift(-h)
```

Without the missing-value check, comparing an unknown future return with zero would produce a false result and incorrectly label the latest rows as downward outcomes. The stored target end date is used to exclude labels that have not matured before a later partition. Rows without targets can still be passed to a trained model for a current forecast.

The execution-price helper reads the unadjusted execution field when shares are calculated. It rejects a missing or invalid price rather than using the analytical close. This is a stricter input requirement, so data generated before the execution field was introduced needs rebuilding.

Target maturity is checked against the next partition boundary rather than only against the row's own date. A twenty-one-session label attached to a late development date can include several test dates. Keeping that row would allow the training objective to use test outcomes even though the row appears to belong to development. Removing such rows changes sample size, which is why the report gives both the preparation counts and the final comparison counts.

The scaler follows the same boundary rule. Its mean and scale are fitted on earlier observations and then applied to later observations without refitting. Refitting on the full panel would reveal the distribution of future inputs. Saving the feature list with the fitted transformation also protects against a quieter failure: valid numerical columns supplied in the wrong order could produce a prediction without raising an exception.

4 3 News preparation and FinBERT

Historical articles are stored by stock symbol with their publication information. Duplicate date-title records are removed. The preparation stage keeps at most five articles per stock per day to limit processing time. This cap can omit information on busy news days and forms part of the experiment definition.

FinBERT processes batches of sixteen items, with text limited to 512 tokens. Scores are cached with a model identifier, text hash and truncation setting. Reusing a cached result is valid only when these inputs match.

The article score multiplies the sign of the predicted class by its confidence. A positive label with confidence 0.80 contributes +0.80; a negative label with confidence 0.70 contributes -0.70; a neutral label contributes zero. This differs from subtracting the probability of negative sentiment from the probability of positive sentiment. Session aggregation records average and weighted scores, class counts, article count and disagreement.

The eight numerical sentiment fields include scores, counts and availability indicators. Fusion can carry score fields and their labels forward for the configured short period, but it does not carry article counts. The no-news flag therefore continues to identify sessions without fresh articles. This avoids overstating coverage merely because a previous score is still available.

The fusion stage also checks whether an input file contains recognised sentiment columns before aggregating it. Files with an incompatible schema produce a logged warning and no-news defaults. A failure during article scoring is separately recorded as a failed job. The distinction between an absent article and an unsuccessful scoring operation is retained in the processing record.

Fusion joins the sentiment fields onto the technical panel by stock and trading session, aligning each score with the price row it sits beside rather than with the raw article timestamp. That session-level join is what lets the combined variant be compared against the technical-only variant on identical rows, and what keeps the no-news flag and any carried-forward score consistent with the label for that session.

4 4 Forecast models and parameter choices

The research XGBoost regressor uses 300 trees, maximum depth five and a learning rate of 0.05. Technical-only, sentiment-only and combined variants share the experiment procedure. Holding the model settings fixed makes the feature comparison easier to interpret, although it does not guarantee that these settings suit each input set equally well.

The application uses a separate XGBoost classifier with a maximum of 600 trees, depth six and learning rate 0.05. It stores its ordered feature list and scaler with the model. An isotonic calibration stage learns a non-decreasing mapping between model scores and observed outcomes from a separate earlier portion of data. The evaluation reports both the raw and calibrated results. Isotonic calibration was preferred over a parametric sigmoid fit because it assumes no fixed shape for the score-to-frequency mapping and can correct non-sigmoidal distortion; the trade-off is that it needs more calibration data and can overfit a small sample.

The LSTM uses thirty-session windows, twenty-four features, two recurrent layers, 128 hidden units and dropout of 0.2. Dropout omits a proportion of internal activations during training to reduce dependence on particular units. Indexed windows and batched inference reduce memory use. The recorded evaluation trained on 899,153 sequences, with a maximum of fifteen epochs and the best validation result at epoch fourteen. An epoch is one pass through the training samples.

The compact experimental GRU uses sixteen hidden units and one time step. A fixed ensemble averages component forecasts. The HMM selector estimates an unobserved market state from observed data and uses development results to choose a component [21]. Its state model excludes the final test period, but its fit is not repeated inside every development fold. These optional components are not scored in the final experiment record, which has deep-model evaluation disabled.

The live regime component fits a three-state Gaussian hidden Markov model to standardised return and volatility features [21]. States receive labels using annualised mean-return bands of plus or minus 5%. The current regime uses posterior probabilities averaged over the latest forty-two observations. A trend-and-momentum heuristic is available when fitting fails; the returned method identifies that fallback. The offline selector uses a separate HMM fit, so sharing a model family does not make the implementations identical. The live market proxy averages price levels, which gives expensive shares greater influence and is not an equal-weight return index.

Model	Task and inputs	Configuration or status
Research XGBoost	Forward return; technical or sentiment	300 trees; depth 5; learning rate 0.05
Deployed XGBoost	Direction; saved feature schema	600-tree cap; depth 6; learning rate 0.05; early stopping 30
Experimental GRU	Return; one observation	One step; 16 hidden units; fixed training budget; not scored in final record
Deployed LSTM	Return; historical sequences	30 sessions; 24 features; 2 layers; 128 hidden units; dropout 0.2
Fixed ensemble	Experimental return forecasts	Equal component weights; not scored in final record
HMM selector	Select experimental predictor	Development state fit; final test excluded; not fully nested; not scored in final record
FinBERT	Article sentiment	ProsusAI/finbert; batch 16; 512 tokens
PPO	Portfolio allocation	3 seeds; 51200 steps; learning rate 0.0003; layers 256/128/64

Table 3 Distinct model responsibilities and configuration provenance. Fixed settings are not evidence of completed tuning.

These are fixed project settings rather than the result of a completed systematic hyperparameter search. A hyperparameter controls the model or training procedure before fitting, such as tree depth or learning rate. Early stopping chooses when to stop within a training run; it does not compare all possible architectures. The saved training configuration records the additional settings without implying that they are optimal.

4.5 Portfolio optimisation and PPO

The Markowitz allocator balances estimated return against a risk-aversion penalty on portfolio variance. A cost-aware variant also penalises turnover, the amount of the portfolio traded. Where sufficient history is available, covariance shrinkage stabilises the covariance estimate by combining the sample estimate with a structured target [22]. A diagonal fallback is available when the fuller estimate cannot be used.

The constraints limit individual positions and retain the minimum cash required by the selected profile. Allocation then converts weights to whole shares and checks affordability after fees. The saved portfolio is updated only after approval. Existing holdings are compared with targets so that a rebalance describes the required changes rather than treating the account as empty.

The PPO environment observes five price-derived measures: twenty-one-session momentum, volatility, Relative Strength Index (RSI), volume ratio and price relative to the twenty-session average. It does not use the XGBoost, LSTM or sentiment predictions in the recorded portfolio comparison. The observation is passed to a policy that proposes an allocation; the environment applies the trading transition and computes a reward after costs.

Three training runs use different random seeds, which control random initialisation and sampling. The learning rate is 0.0003 and each rollout contains 2,048 steps. The requested 50,000 steps become 51,200 recorded steps per seed because complete batches are collected. Results from all three seeds are retained.

The allocation objective can be read as estimated portfolio return minus a risk penalty and, for the cost-aware variant, a trading penalty. The return term is the weighted sum of asset return estimates. The variance term combines pairs of weights with the corresponding covariance entries. This makes diversification part of the calculation: concentrating in assets that move together increases the joint risk term. Constraints are applied alongside the objective rather than checked only after an unconstrained solution has been produced.

The historical Markowitz path first screens for top-K stocks with positive trailing returns, retaining relevant existing holdings. Optimisation therefore operates after a momentum-based selection stage. Any observed gain belongs to that complete procedure, not solely to covariance optimisation. A direct test of the optimiser's contribution would hold the selected basket fixed and compare alternative weight calculations on it.

In the PPO environment, negative action values are clipped, stock and cash weights are normalised, and an all-zero action becomes cash. The reward divides net step return by a recent-return standard deviation with a lower bound of 0.001. That bound prevents division by a near-zero value but also influences the reward scale. Cash earns zero in the simulation. These assumptions matter when interpreting a policy that prefers cash or repeatedly changes small positions.

Transaction costs are based on the difference between target equity weights and the previous portfolio's drifted weights. After a price movement, those weights no longer equal the last requested allocation. Updating the drift prevents the next cost calculation from assuming that the portfolio rebalanced itself for free. Terminal equity liquidation is charged separately. The environment's return update is therefore an accounting transition as well as a learning step.

4 6 Explanation and avatar integration

The SHAP exporter loads the saved classifier, scaler and feature order. It selects the latest complete row for each stock, up to 500 rows, and produces a global importance plot and a local waterfall. A saved record identifies the sample and selected stock. The explained output is the classifier's raw margin before calibration, expressed in log-odds rather than percentage return.

Ollama receives structured evidence and returns a natural-language explanation. A check rejects unsupported numerical claims, and a fixed template provides a fallback. This check can catch invented amounts but cannot prove that every interpretation is correct; the explanation still needs to preserve the meaning of the supplied evidence.

The LiveAvatar backend creates a short-lived session token. The account API key stays on the server. The avatar is configured without its own conversational context, and the browser uses the repeat-text operation for the reply already produced by the chatbot [23]. Long replies are sent in chunks. Microphone transcripts are returned to the shared chat handler, duplicate transcript events are ignored and leaving the page ends the session.

Mocked avatar tests check reply content, chunk completion, microphone state and cleanup. They test the local integration without contacting the provider. Successful live audio/video and Ollama generation remain unverified in the available record.

4 7 Interface behaviour and error handling

Imported spreadsheet values are escaped before they enter HyperText Markup Language (HTML), including values placed in input attributes. Corrupt spreadsheet containers are handled as invalid uploads instead of uncaught server errors. The user can correct the input without losing access to other views.

The Dashboard shows saved cash and holdings. It does not draw a forecast as realised growth. The Stock Predictor labels the current LSTM estimate separately from its historical XGBoost comparison and states the horizon in trading sessions. The interface offers one, five and twenty-one trading sessions, enabling horizons with available model artifacts rather than rescaling a single forecast.

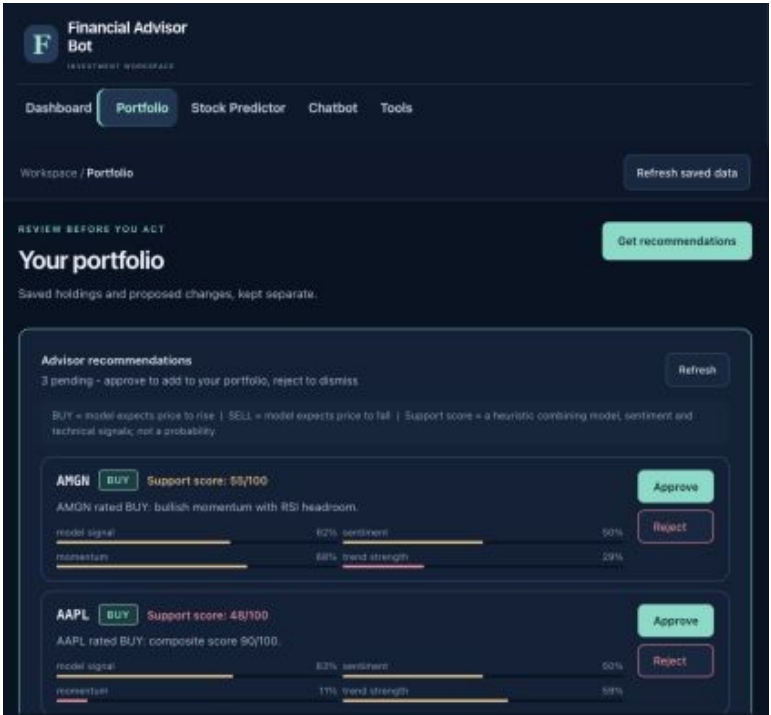


Figure 4 Application screenshot of portfolio review. Proposed changes are separate from saved holdings and require approval.

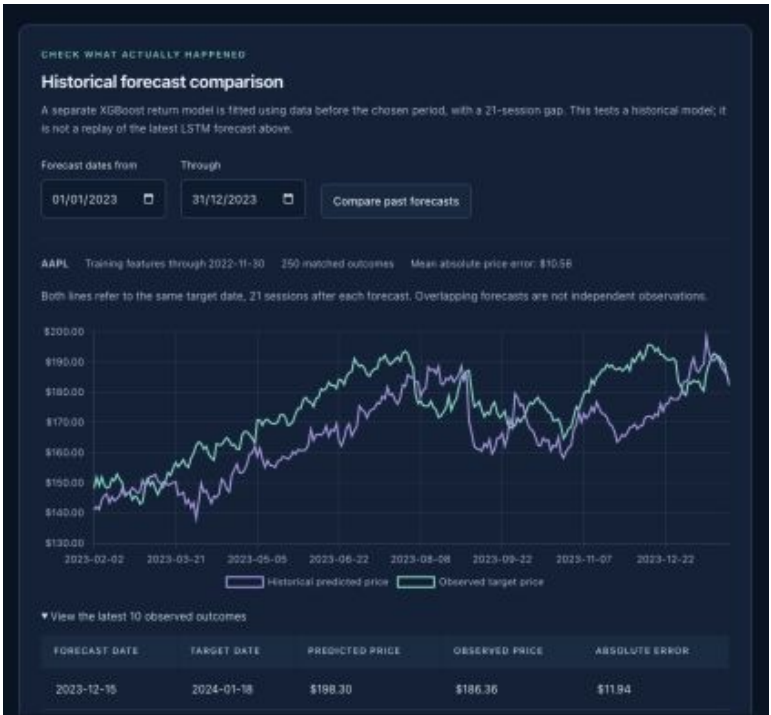


Figure 5 Application screenshot of a historical prediction comparison. The historical XGBoost evaluation is labelled separately from the latest LSTM forecast.

4 8 Testing and computation

The latest regression run covers the frontend, avatar and portfolio checks. Tests cover date alignment, financial constraints, malformed imports and missing values. The remaining warnings concern dependency deprecations and unbounded observation-space limits in the reinforcement-learning environment.

An earlier native Python crash during backtesting was investigated through its crash report. Startup configuration limits Open Multi-Processing (OpenMP) threads before numerical libraries are imported. Fresh-process checks and targeted backtest requests supported this mitigation. The project environment is used consistently because the separate base environment previously lacked plotting dependencies.

Tree prediction cost grows with the number and depth of trees. Sequence prediction also grows with window length, while storing a covariance matrix grows with the square of the asset count. Reusing compatible features and model files reduces repeated work. The project uses library implementations for the learning algorithms; its own logic coordinates data timing, accounting, evaluation and the user-facing workflow.

4 9 Recommendation queue and import state

The recommendation service ranks eligible BUY and SELL signals and keeps a queue, with up to three visible at once. Approving or dismissing a proposal reveals another. Generate new excludes the currently visible stock symbols; it does not guarantee that a finite candidate universe will never repeat. Both actions undergo a simulated transaction against current cash, holdings and risk limits.

When fewer than three eligible signals remain, the interface shows fewer proposals. It does not turn HOLD into BUY or invent a sale to fill the screen. Approval rechecks the transaction under the store lock. Manual trades clear outdated proposals, and Build rejects a changed portfolio rather than overwriting a concurrent update.

Each queued idea also carries a confidence score that combines the model's probability of an upward move, the size of the predicted return and a small set of quality factors, so a large predicted return paired with a weak or uncertain probability does not present as high confidence. The score is shown alongside the recommendation rather than folded into a single opaque rank.

Import preview validates the file before holdings are saved. The form loads the existing risk profile and budget rather than restarting with defaults. Repeated profile creation now preserves the existing account, preventing a failed import from first erasing holdings and recommendation history. A successful import replaces the holdings with the reviewed file contents; it is not an implicit merge of two portfolios.

5 Evaluation

Chapter word count 2,291

5 1 Evaluation approach

Evaluation covers the dataset, forecasts, portfolio simulations, explanations and software behaviour. The saved results use the corrected next-close execution procedure. Earlier runs exposed timing and benchmark problems, so the figures below replace the earlier return summaries.

The final periods have been inspected more than once during development. Comparisons are still useful for describing the recorded configurations, but a new configuration chosen after seeing these results would require a new holdout for independent confirmation [16]. No performance claim is based on training reward or a model fitted to the complete historical period.

5 2 Dataset coverage

The requested window is 2010 to 2023. Usable features begin on 18 October 2010 after data availability and indicator warm-up. The deployed evaluation loads 565 stocks. Historical membership filtering retains 1,312,195 of 1,757,407 rows before the final target filters. This removes out-of-membership observations, although unavailable histories for delisted stocks can still bias the sample.

Horizon	Prepared panel rows	Tickers	Final test decision dates
1 session	1319726	549	11 Sep 2020 to 28 Dec 2023
5 sessions	1317822	549	8 Sep 2020 to 21 Dec 2023
21 sessions	1310219	547	20 Aug 2020 to 29 Nov 2023

Table 4 Prepared research panels before the final-test split. Final-test samples contain 381143, 380559 and 378236 observations respectively; panel rows are not independent trials.

The wider fusion audit contains 629 successful stocks, eight failures and 1,882,190 stock sessions. News is observed in 527,014 sessions across 451 stocks, approximately 28.0% of the total. The input directory contains 1,140,774 prepared articles in 457 files. That count describes prepared input, not a reconciled count of successful sentiment classifications. Failed symbols remain documented in the fusion audit.

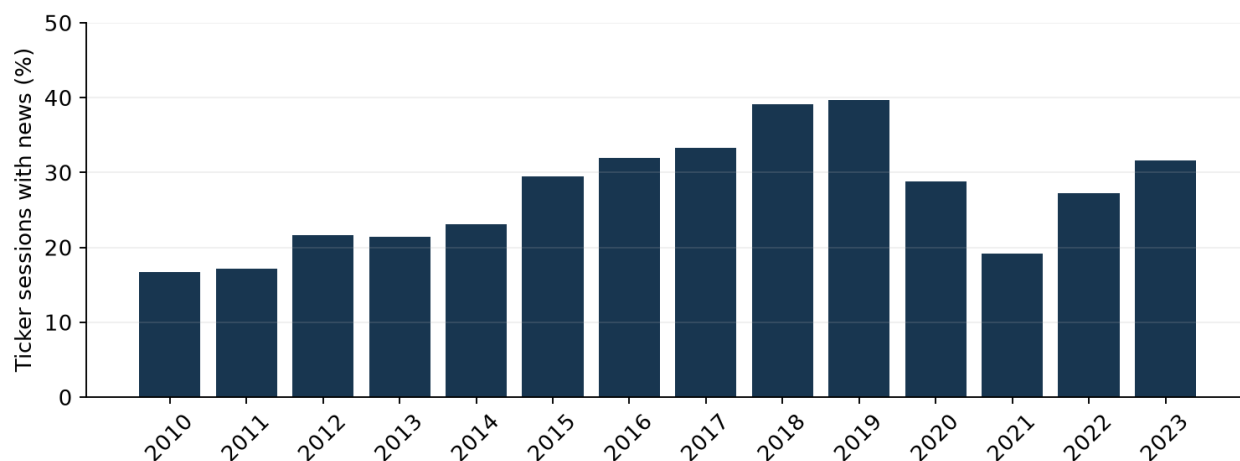


Figure 6 Observed-news coverage in the fusion audit, 2010 to 2023. The denominator is ticker-sessions, not articles or unique trading days.

Coverage rises from 16.7% in 2010 to 39.7% in 2019, then falls to 19.2% in 2021. A weak aggregate sentiment result may therefore combine several situations: useful news, neutral news, carried scores and no available text. Performance within news-bearing sessions would be a more direct follow-up comparison, provided the technical baseline uses those same sessions.

The deployed split has 2,456 development dates and 825 test dates, with a twenty-one-session gap. Test decisions run from 20 August 2020 to 29 November 2023. Models are compared on common stock-date rows. The horizon-specific research panels have different final dates because later outcomes must be available for each target.

5 3 Forecast accuracy and baselines

Balanced accuracy averages the proportion of upward outcomes identified correctly and the proportion of downward outcomes identified correctly. A classifier that always predicts up scores 0.50. Ordinary accuracy would give that same classifier 54.70% in this test because upward outcomes are more common.

Area Under the receiver operating characteristic Curve (AUC) measures how well scores separate the two classes across thresholds. A value near 0.50 indicates little separation. Mean Absolute Error (MAE) measures the average size of a return-prediction error; Root Mean Squared Error (RMSE) gives more weight to large errors. The Information Coefficient (IC) is the Spearman rank correlation between forecasts and outcomes.

Classifier	Balanced accuracy	AUC
Always up	0.5000	Not applicable
Logistic technical	0.4989	0.5072
XGBoost technical	0.4989	0.5025
XGBoost combined	0.4997	0.5032

Table 5 Deployed classifier evaluation on 378236 test rows. Balanced accuracy and AUC measure different properties.

The technical XGBoost classifier scores 0.4989 balanced accuracy and 0.5025 AUC. Adding sentiment raises these to 0.4997 and 0.5032, still close to the constant baseline. Logistic

regression has the largest AUC, 0.5072, despite being the simpler model. The present results give little reason to prefer the additional tree complexity for directional classification. The logistic baseline is fitted with a Newton-cholesky solver on standardised inputs, so its near-baseline score reflects the weak linear signal in the features rather than an optimisation failure.

Model	MAE	RMSE	Direction
Zero return	0.068595	0.091833	Not meaningful
Historical mean	0.068176	0.091077	0.5471
XGBoost	0.068963	0.092111	0.5276
LSTM	0.068124	0.090659	0.5472

Table 6 Return regression on the same 376641 sequence-eligible test rows. Errors are decimal returns; direction is ordinary accuracy.

The LSTM requires thirty historical rows and excludes 1,595 otherwise eligible observations. All regression baselines in this table use its remaining 376,641 observations. Its MAE is 0.068124, compared with 0.068176 for the historical mean. The difference is 0.000052 in decimal return, or 0.0052 percentage points. Its ordinary direction accuracy is marginally higher, 0.5472 against 0.5471. The small error reduction has no paired uncertainty interval, so the ranking is descriptive.

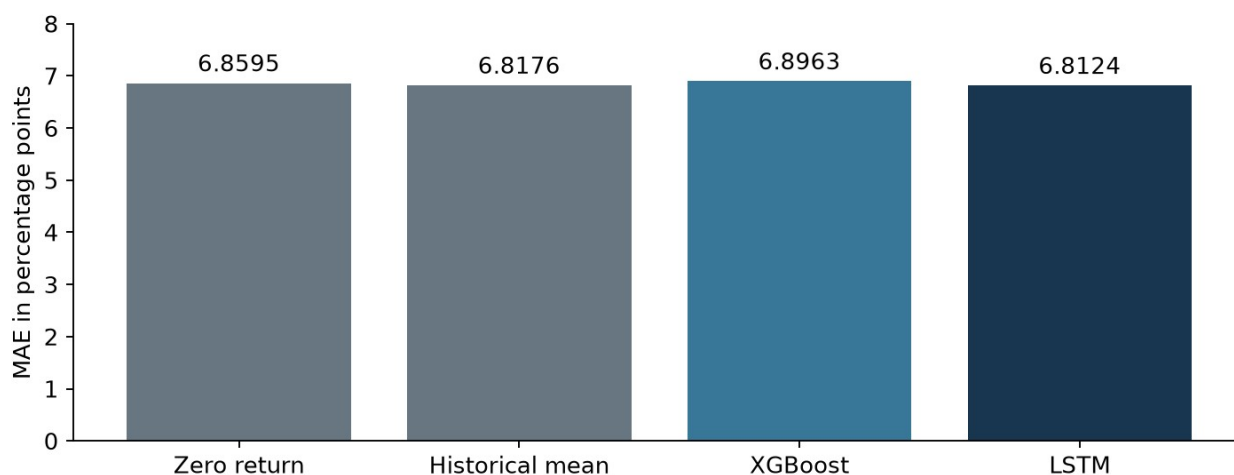


Figure 7 Matched 21-session return errors on 376641 rows. The small LSTM advantage is descriptive; no paired interval is available.

The XGBoost regressor has larger MAE and RMSE than both simple return baselines. This suggests that its fitted variation adds error on this test sample. The zero-return model has no positive or negative forecast, so its direction score is not used as an accuracy benchmark. The evaluated baselines are listed in the table; an AutoRegressive Integrated Moving Average (ARIMA) model was not evaluated.

5 4 Sentiment and model combinations

The research models predict returns and are separate from the deployed direction classifier. Table 7 uses C for technical-only XGBoost, D for sentiment-only and E for the combined model. Their settings and comparison procedure remain fixed within each horizon.

Horizon	Model	Balanced accuracy	IC	Net return %
1	C	0.5035	0.0112	-41.22
1	D	0.5005	0.0006	-61.81
1	E	0.5034	0.0106	-28.37
5	C	0.4988	0.0091	82.74
5	D	0.5003	0.0002	115.49
5	E	0.4992	0.0104	63.46
21	C	0.4952	0.0234	120.20
21	D	0.4999	0.0008	68.35
21	E	0.4956	0.0226	102.46

Table 7 Research regressors on matched final-test observations within each horizon. C uses technical features; D sentiment; E both.

Adding sentiment changes balanced accuracy by -0.0001, +0.0004 and +0.0004 at one, five and twenty-one sessions. These changes are small and do not match the original expectation of a substantial improvement. Sentiment-only rank correlations are close to zero at all three horizons.

The paired deployed comparison estimates an ordinary directional-hit improvement of 0.000801. Its 95% Confidence Interval (CI) runs from -0.000264 to +0.001735 and includes zero. Expressed as percentage points, this is an estimated gain of 0.0801 with an interval from -0.0264 to +0.1735. It concerns ordinary directional hits rather than balanced accuracy.

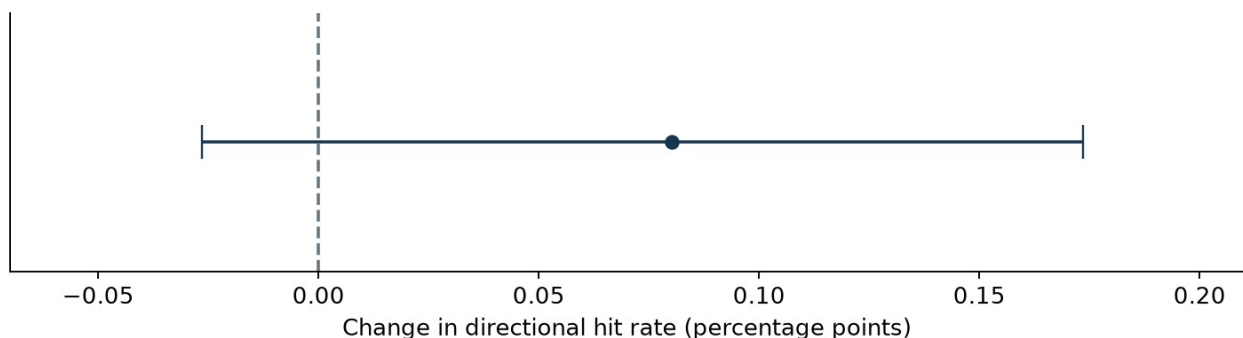


Figure 8 Paired sentiment contribution to the deployed directional hit rate. The 95 percent date-block interval crosses zero. This is not a balanced-accuracy interval.

The five-session research MAE comparison has a small positive interval, from 0.000004 to 0.000034, around an improvement of 0.000019. Several horizons and metrics were examined without adjusting for multiple comparisons. This makes the isolated positive result less convincing than a preselected test with the same interval. Resampling blocks of twenty-one dates accounts for some dependence between nearby outcomes, but sensitivity to block length remains untested.

The final research record has optional deep-model evaluation disabled. It contains no GRU, fixed-ensemble or HMM-selector scores. Their implementation is described in Chapter 4, but they

cannot support a numerical comparison in this evaluation. Restoring older scores would mix configurations and evidence versions.

5.5 Calibration and weak predictive performance

The Brier score measures squared probability error. Expected Calibration Error (ECE) compares predicted probabilities with observed frequencies within groups. Calibration reduces Brier score from 0.25071 to 0.25062 but increases ten-bin ECE from 0.03593 to 0.05205. The method has therefore improved one measure slightly while worsening the other. The result depends on the calibration sample and grouping scheme [14].

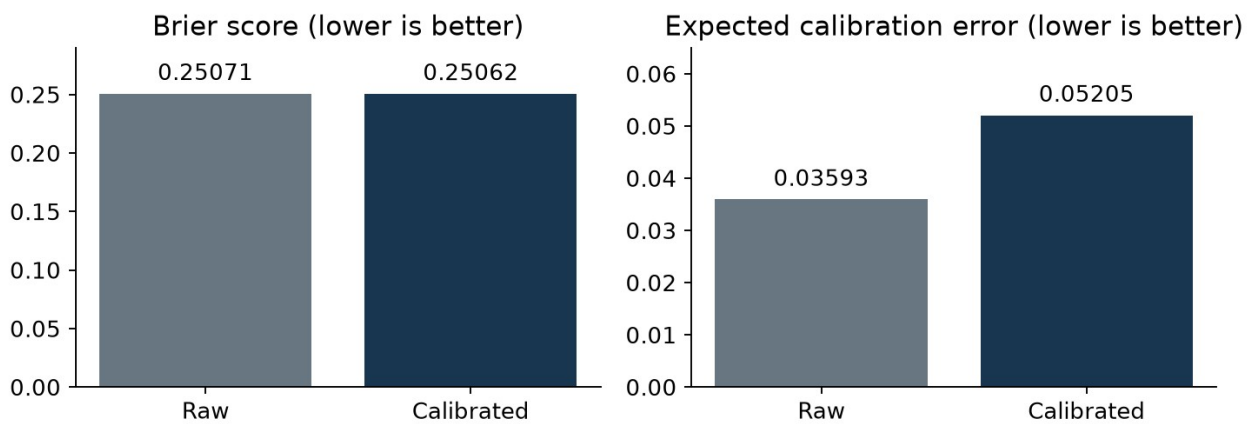


Figure 9 Calibration metrics on final-test observations. Lower is better for both measures. Brier improves slightly while expected calibration error increases.

Feature removal provides more specific evidence about the weak forecasts. Removing volatility improves balanced accuracy from 0.4952 to 0.5024 and IC from 0.0234 to 0.0391. Removing trend worsens accuracy. This is consistent with some input groups being redundant or poorly suited to the fitted model, but it does not identify a universally better feature set because the comparison has already been observed.

Feature set	Balanced accuracy	IC	Delta
Full technical	0.4952	0.0234	+0.0000
Without trend	0.4926	0.0224	-0.0026
Without momentum	0.4939	0.0197	-0.0013
Without volatility	0.5024	0.0391	+0.0072
Without volume	0.4968	0.0242	+0.0016
Without strength	0.4966	0.0253	+0.0014

Table 8 Exploratory 21-session ablation. Delta is the ablated balanced accuracy minus the full technical model, so positive means removal improved the metric.

Sparse news is another plausible contributor: around 72% of fusion sessions have no observed fresh article. The current signed sentiment score also discards probability information by keeping only the predicted class and its confidence. These properties could weaken a useful textual signal

before it reaches the return model. A matched news-only analysis and an alternative score definition would test those explanations more directly.

The near-baseline results may also reflect changing relationships between indicators and future returns. No regime-specific error analysis establishes that explanation here. Fixed parameters and the lack of a systematic tuning study limit what can be concluded about the algorithms themselves. The experiments show weak performance for these configurations, not that financial prediction is impossible.

5.6 Portfolio performance

The portfolio script reserves the final 25% of the common trading calendar for testing, leaving the earlier 75% for PPO training. This is a chronological division of dates, not a random quarter of stock rows or the full 2010 to 2023 forecasting panel. Intersecting the histories of 563 assets produces the shared calendar and places the split on 22 August 2023. Common dates allow both methods to observe the same market, but selecting assets with complete histories can introduce selection bias.

The script then truncates the end to complete twenty-one-session holding periods, allowing one session for next-close execution. Return targets require later prices, leaving 65 eligible decision dates through 21 November and outcome prices through 21 December. Markowitz rebalances every twenty-one decision sessions, selecting offsets zero, twenty-one, forty-two and sixty-three: four rebalances. PPO can act daily. The 25% rule therefore does not imply that both methods make the same number of trades or have equally sampled risk statistics.

Method	Net return %	Comparison constraint
Fixed buy-and-hold	Unavailable	Missing execution prices
Equal-weight top-K	2.37	Unconstrained selected basket
Historical Markowitz	10.41	Momentum screen; four rebalances
Cost-aware Markowitz	10.36	Turnover penalty
Smart rebalance	10.36	Rebalance rules
PPO seed 0	9.99	Daily policy
PPO seed 1	14.98	Daily policy
PPO seed 2	7.77	Daily policy

Table 9 Recorded portfolio endpoint returns after costs. Fixed buy-and-hold is unavailable. Selection, trading frequency and constraints differ; the rows do not provide a controlled algorithm ranking.

Markowitz returns 10.41% after recorded costs. The cost-aware and smart variants both return 10.36%, compared with 2.37% for selected equal-weight top-K. Smart rebalance has lower recorded turnover than cost-aware allocation despite the same rounded endpoint return. Equal final wealth does not imply equal trades or risk along the path.

PPO returns 9.99%, 14.98% and 7.77% across its three seeds, averaging 10.91%. Fixed buy-and-hold is unavailable because required execution prices are missing for five stocks. The recorded 6.93% periodic equal-weight gross return is a different strategy and cost basis. Neither number

justifies a claim that PPO beats buy-and-hold. Selecting only the strongest seed would also hide substantial training sensitivity.

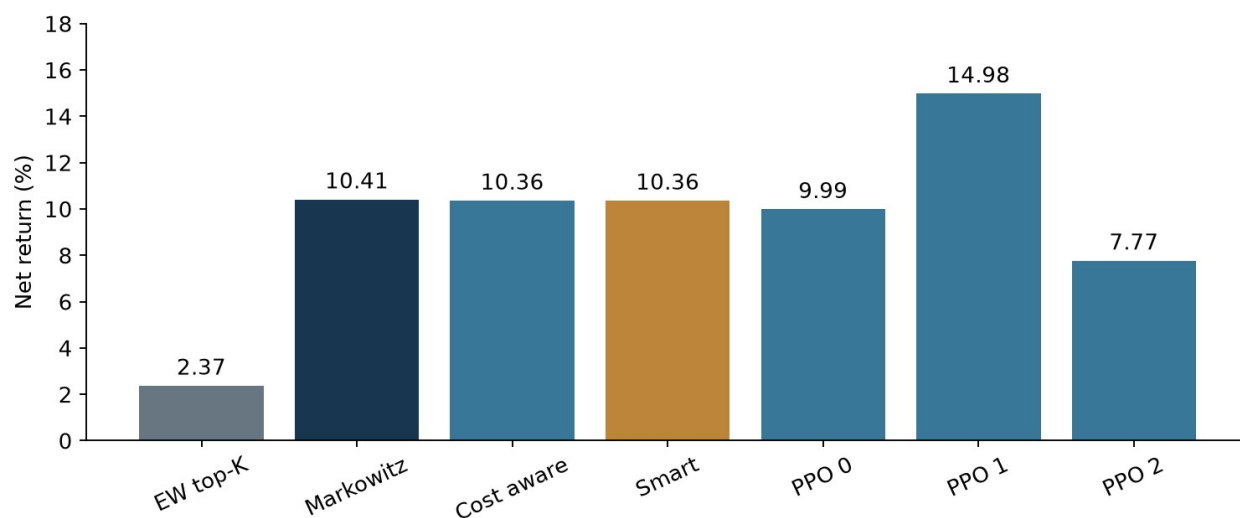


Figure 10 Saved portfolio endpoint returns, net of the reported costs. Trading frequency, constraints and selection differ; the bars do not establish a general strategy ranking.

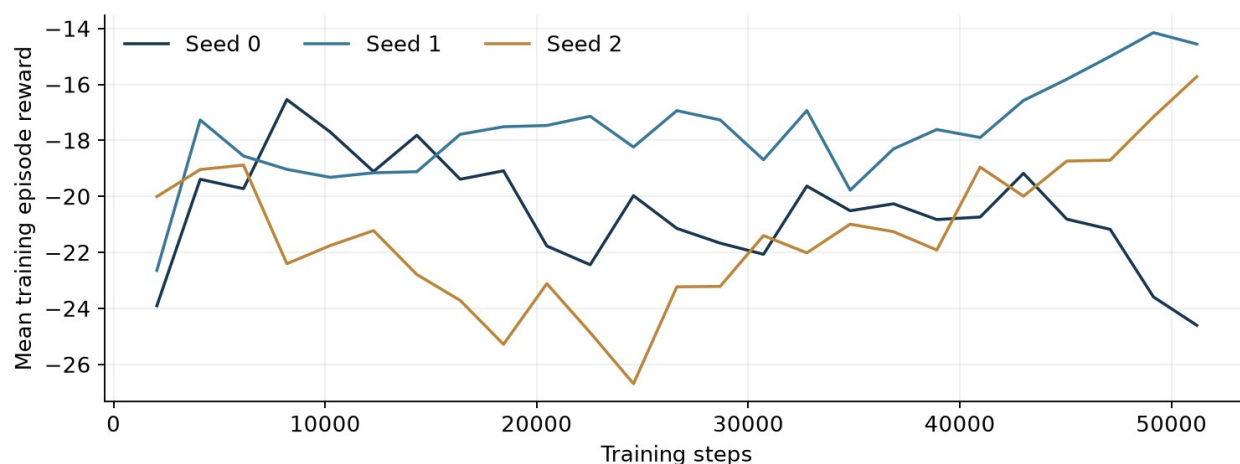


Figure 11 Actual PPO training reward records for three seeds. Training reward is not a test return, and the seeds share one economic history.

The Markowitz result needs position-level attribution before a strong economic claim is made. Four holding periods are too few to show stable behaviour across market conditions. Common-history selection can also favour assets whose data remain available over the complete window. Daily PPO risk statistics and horizon-sampled Markowitz statistics are not directly comparable.

Execution takes place at the next observed close after the signal. The cost rate is ten basis points, or 0.10%, per unit traded. The simulation does not fully model spreads, market impact or delisting settlements. Equal-weight top-K is an unconstrained selected basket, not a risk-matched comparator for every allocation. These differences restrict what the return ranking can establish.

Research XGBoost trading returns also vary sharply by horizon, from -41.22% at one session to +120.20% at twenty-one sessions. A fixed buy-and-hold comparison is unavailable in those panels because required execution prices are missing. The recorded periodic equal-weight gross series is a different strategy. Positive returns alongside weak prediction can arise from exposure, position sizing and selection, but the current aggregate results do not identify their separate contributions.

The forecasting test sample has an average forward return of 0.01173, separately from the shorter portfolio comparison. This is descriptive evidence of positive average outcomes, not an input available to the model beforehand. Long-only exposure can benefit in such a sample without accurately distinguishing winners from losers. Conversely, near-random direction does not prove the precise cause of an allocator's profit. Attributing returns to market exposure, selection or concentration requires corresponding holdings and contribution analysis.

5.7 Explanation outputs

The global SHAP plot uses 500 latest complete observations, one per selected stock. Accumulation/distribution has the largest mean absolute contribution, followed by Average True Range (ATR) and Moving Average Convergence Divergence (MACD). These price and volume measures dominate the displayed individual features. The grouped remainder prevents a precise statement about every feature outside that list.

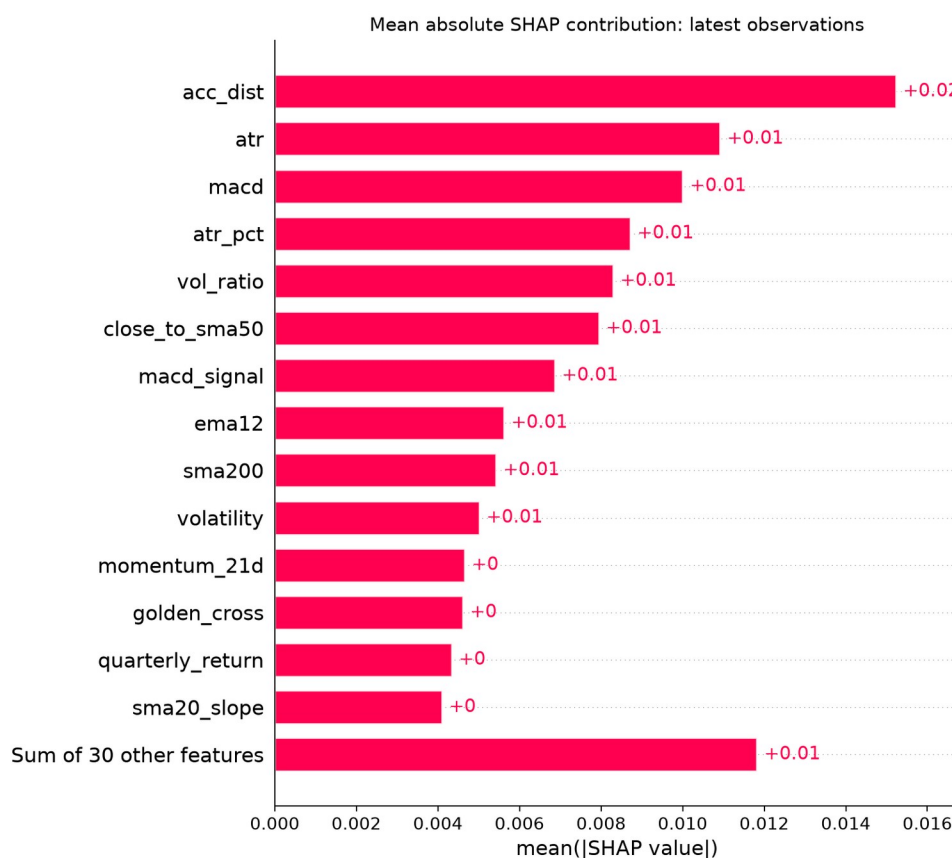


Figure 12 Actual SHAP export from the deployed classifier, using 500 latest complete observations. Contributions are raw classifier margins before calibration. Source: *shap_provenance.json*.

The local American Airlines (AAL) example uses the latest complete row for that stock. Its raw margin moves from approximately 0.337 to 0.276. Accumulation/distribution contributes about -0.04 and price relative to the fifty-session average about -0.02. Smaller positive contributions partly offset them. The horizontal axis is the classifier margin, not a predicted percentage return. Feature values shown on the left have been transformed for the model.

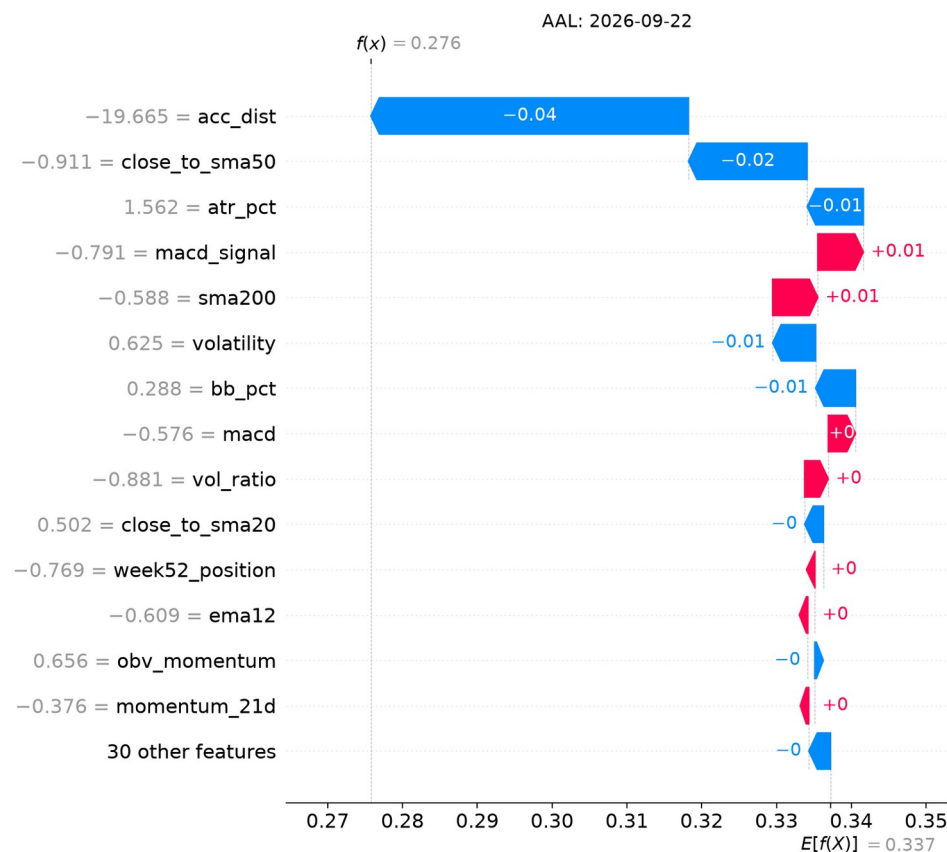


Figure 13 Actual AAL explanation for the latest complete AAL row. The reference margin is about 0.337 and the output about 0.276. Feature values are scaled inputs; contributions are not returns.

This explains one model response and its broader feature-use pattern. It does not demonstrate why the historical return model performed poorly: the SHAP sample uses a different model and later observations than the feature-removal experiment. The appropriate use is to check what the application is relying on and whether the explanation describes those inputs correctly.

5 8 Software and interface review

The regression checks cover malformed spreadsheets, escaped input values, date alignment and portfolio accounting. The final run passes 151 Python tests with four warnings. Frontend and avatar contract checks also pass, and portfolio verification passes eighteen checks. Two warnings concern deprecated dependency interfaces and two concern unbounded observation-space limits in the reinforcement-learning environment.

The supplied interface feedback reported that the initial screen contained too much information. The revised five-view layout separates account information, portfolio decisions, forecasts, conversation and extra tools. Dashboard opens first, and proposals remain in Portfolio until approved. The screenshots show the resulting layout. There are no measured completion times or participant scores for the revised version.

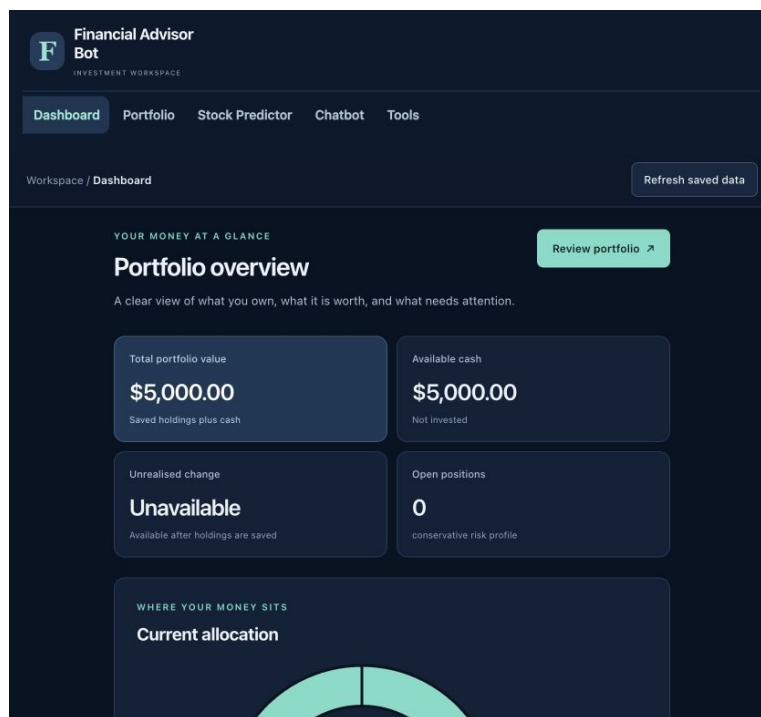


Figure 14 Application screenshot showing saved cash and no holdings. An unavailable gain is not replaced by an invented growth history.

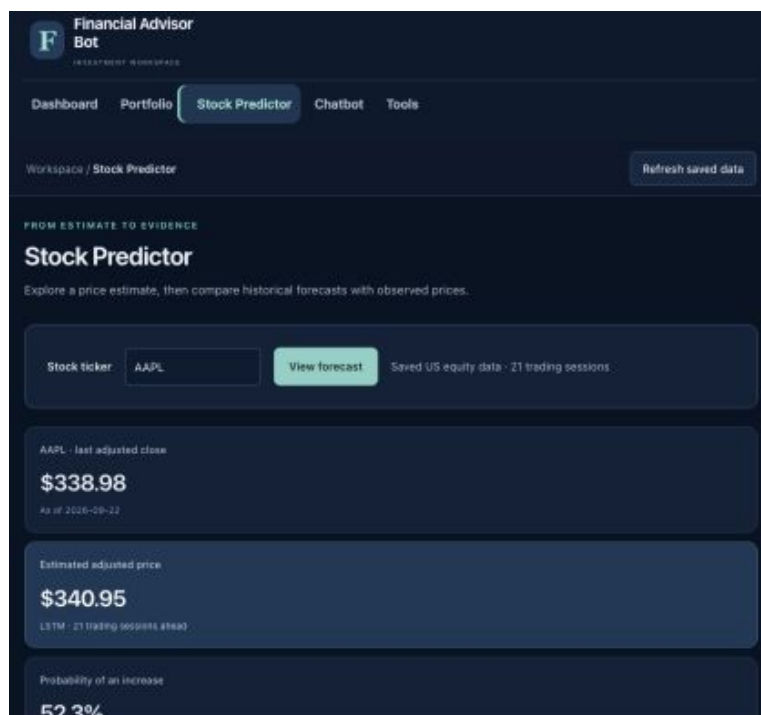


Figure 15 Application screenshot of the stock predictor. The LSTM price estimate and directional classifier output have different meanings.

The avatar offers an alternative for people who prefer speaking or have difficulty with text input. Keeping the written answer alongside speech also supports review of financial figures. Its local tests use a simulated provider, so they do not establish live call quality or disability access. Ollama generation and LiveAvatar audio/video still require verified live sessions. These gaps remain separate from the passing calculation and interface checks.

5.9 Validity and unanswered questions

The results support different conclusions for the original questions. Learned direction models do not clearly improve on the constant baseline. The matched LSTM comparison shows a very small reduction in return error. Sentiment has no reliable directional advantage in the reported paired interval. Allocation methods produce positive short-window returns, but the missing fixed benchmark and unequal rules prevent a clean claim of superiority.

Internal validity is limited by repeated inspection of the final period and incomplete attribution of portfolio returns. External validity is limited by one equity market, unavailable delisted histories and the short allocation window. Measurement validity is limited by the distinction between forecast accuracy, trading returns and user understanding. Passing a calculation test answers none of the questions about whether a beginner interprets that calculation correctly.

The software evidence covers selected behaviours rather than every executable line. No code-coverage percentage or measured response-time distribution accompanies the test count. Live conversation remains a separate acceptance gap. A future check should establish that the displayed answer, spoken answer and session state agree during provider delays and interrupted microphone input, while retaining the successful financial checks. The added transaction tests check fee reconciliation, invalid quantities, missing execution prices, position limits after adding to an existing holding, and conflicting updates. These failures affect the saved account even when every forecast is numerically valid. Testing them separately avoids using prediction accuracy as a substitute for application correctness. A larger recommendation queue improves navigation, but does not create additional investment evidence. Requiring three different trades regardless of available signals would reward activity over suitability. The interface therefore separates a user-requested purchase from a model proposal and preserves the option to make no transaction when constraints prevent execution.

The latest checks also cover repeated profile creation, failed imports and recommendation queue advancement. These target the boundary between displayed advice and saved holdings, where a successful forecast calculation alone would not prevent state loss.

6 Conclusion

Chapter word count 836

The Financial Advisor Bot combines price and news preparation, forecasting, constrained portfolio proposals and explanations. It supports a new investor entering a budget and an existing investor importing holdings. The revised interface separates account information, proposals, predictions, conversation and tools. Approval is the required step between a recommendation and the saved portfolio.

The first research question concerned whether learned models improve on simple forecasts. The directional classifiers remain close to the always-up balanced-accuracy baseline. The LSTM reduces matched mean absolute error by 0.000052 in decimal return relative to the historical mean. That difference is too small, without an uncertainty interval or economic analysis, to justify describing the network as a substantially better predictor. The simpler baseline is a substantive result of the project, not a comparison to leave out.

The second question concerned the contribution of sentiment. Adding news produces small changes in the recorded metrics, and the deployed directional-hit interval includes zero. The isolated positive five-session absolute-error interval does not establish a general advantage across horizons. Sparse coverage, article aggregation and the score definition offer plausible explanations to investigate, but none has been isolated experimentally. The present evidence supports retaining sentiment as an exploratory input rather than presenting it as a proven improvement.

The portfolio question is only partly answered. Markowitz and PPO produce positive recorded returns, with a 10.41% Markowitz endpoint and a 10.91% mean across PPO seeds. Their selection stages, trading frequencies and risk measurements differ. Fixed buy-and-hold cannot be evaluated from the required prices in the saved comparison. These limitations prevent a claim of market outperformance or a general ranking of the algorithms. The large spread across the PPO seeds is itself evidence about reliability.

6 1 Contribution

My contribution is the decision-support workflow joining dated news, price features, model inference, constrained share allocation and reviewable explanations. I implemented the timing and accounting controls that allow these components to operate together: analytical and execution prices are separated, label maturity determines eligibility, sequence comparisons use matched observations, and share rounding preserves cash after fees. These controls address difficulties I encountered as a beginner, particularly treating a confident-looking prediction as a complete investment decision. The project also connects numerical explanations to written and spoken interaction through a shared answer. Its contribution is this implemented combination and its documented evaluation, rather than a new learning algorithm. The weak forecast results help define where that workflow needs restraint: explanation and correct accounting cannot substitute for reliable prediction.

The avatar provides a second way to receive the chatbot's answer. It is an accessibility-oriented design choice for people who prefer speaking or find text interaction difficult. Its implemented connection and simulated-provider tests do not establish live reliability or benefits for disabled users. The report therefore separates the design intention from the evidence needed to demonstrate that intention in practice.

6 2 Future work

Further work should begin with a frozen evaluation protocol and a previously unused period. The same eligible basket, cost assumptions and daily valuation would make portfolio comparisons

more informative. A paired error interval would clarify the LSTM result, while a matched news-bearing subset would help test sentiment without changing the comparison sample. Optional GRU and HMM modules would need reproducible results before their additional complexity could be assessed.

For the application, the next priority is observation of real tasks: importing holdings, interpreting the forecast horizon and deciding whether to approve a proposal. Recorded errors and misunderstandings would guide changes more effectively than adding another screen. The application remains an educational prototype.

The next modelling experiment should compare recurrent forecasts with a linear model using the same lagged inputs. That comparison would isolate whether the network gains from nonlinear memory rather than simply receiving a longer history. Model selection should use development data only, followed by one evaluation on a previously unused period. A forecast improvement must also survive costs before it can justify a transaction rule.

For portfolio evaluation, I would retain a complete daily holdings ledger and report each stock's contribution to wealth. This would distinguish profitable selection from broad market exposure or a single large winner. A common rebalancing schedule, identical cash constraints and an executable fixed-basket benchmark would make the comparison more controlled. Market coverage should expand through documented historical membership, rather than removing difficult stocks merely to improve the result. Expanding beyond the S&P 500 would reuse the same feature, prediction and accounting interfaces. It would still require compatible price fields, exchange calendars, currency handling and historical eligibility data. Models would need retraining and evaluation on the broader universe. More assets increase sequence processing and covariance storage, so unchanged core logic does not imply unchanged runtime or transferable accuracy.

Accessibility requires direct evidence beyond an avatar being present. A small, consented task study could compare text and voice for checking a forecast, finding available cash and rejecting an unsuitable proposal. Completion errors, misunderstood numbers and user preference should be recorded separately. The aim would be to learn when voice helps and when visible text remains essential, rather than assuming that every disabled user benefits from the same interface.

References

- [1] Brad M. Barber, Xing Huang, Terrance Odean and Christopher Schwarz. Attention-Induced Trading and Returns: Evidence from Robinhood Users. *The Journal of Finance*, 77(6), 3141-3190, 2022. <https://doi.org/10.1111/jofi.13183>
- [2] Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. *Proceedings of KDD*, 785-794, 2016. <https://arxiv.org/abs/1603.02754>
- [3] Shihao Gu, Bryan Kelly and Dacheng Xiu. Empirical Asset Pricing via Machine Learning. *The Review of Financial Studies*, 33(5), 2223-2273, 2020. <https://doi.org/10.1093/rfs/hhaa009>
- [4] Sepp Hochreiter and Jurgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780, 1997. <https://www.bioinf.jku.at/publications/older/2604.pdf>
- [5] Thomas Fischer and Christopher Krauss. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654-669, 2018. <https://doi.org/10.1016/j.ejor.2017.11.054>
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT*, 4171-4186, 2019. <https://aclanthology.org/N19-1423/>
- [7] Dogu Araci. FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. *arXiv:1908.10063*, 2019. <https://arxiv.org/abs/1908.10063>
- [8] ProsusAI. FinBERT model card. Hugging Face. <https://huggingface.co/ProsusAI/finbert>
- [9] Zihan Dong, Xinyu Fan and Zhiyuan Peng. FNSPID: A Comprehensive Financial News Dataset in Time Series. *arXiv:2402.06698*, 2024. <https://arxiv.org/abs/2402.06698>
- [10] Harry Markowitz. Portfolio Selection. *The Journal of Finance*, 7(1), 77-91, 1952. <https://doi.org/10.1111/j.1540-6261.1952.tb01525.x>
- [11] Richard O. Michaud. The Markowitz Optimization Enigma: Is Optimized Optimal? *Financial Analysts Journal*, 45(1), 31-42, 1989. <https://doi.org/10.2469/faj.v45.n1.31>
- [12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford and Oleg Klimov. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>
- [13] Scott M. Lundberg and Su-In Lee. A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems* 30, 4765-4774, 2017. <https://arxiv.org/abs/1705.07874>
- [14] Alexandru Niculescu-Mizil and Rich Caruana. Predicting Good Probabilities with Supervised Learning. *Proceedings of ICML*, 625-632, 2005. <https://doi.org/10.1145/1102351.1102430>
- [15] Sayash Kapoor and Arvind Narayanan. Leakage and the Reproducibility Crisis in Machine-Learning-Based Science. *Patterns*, 4(9), 100804, 2023. <https://doi.org/10.1016/j.patter.2023.100804>
- [16] David H. Bailey, Jonathan M. Borwein, Marcos Lopez de Prado and Qiji Jim Zhu. The Probability of Backtest Overfitting. *The Journal of Computational Finance*, 20(4), 39-69, 2017. <https://www.davidhbailey.com/dhbpapers/backtest-prob.pdf>
- [17] Wealthfront. Classic Portfolio Investment Methodology White Paper. Official methodology documentation. <https://research.wealthfront.com/whitepapers/investment-methodology/>
- [18] Bloomberg. Portfolio and Risk Analytics. Official product documentation. <https://professional.bloomberg.com/products/bloomberg-terminal/portfolio-analytics/>
- [19] TradingView. Paper trading main functionality. Official support documentation. <https://www.tradingview.com/support/solutions/43000516466-paper-trading-main-functionality/>
- [20] Portfolio Visualizer. Backtest Portfolio Asset Allocation. Official tool documentation. <https://www.portfoliovisualizer.com/backtest-portfolio>

- [21] Lawrence R. Rabiner. A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition. Proceedings of the IEEE, 77(2), 257-286, 1989.
<https://www.cs.cornell.edu/courses/cs481/2004fa/rabiner.pdf>
- [22] Olivier Ledoit and Michael Wolf. A well-conditioned estimator for large-dimensional covariance matrices. Journal of Multivariate Analysis, 88(2), 365-411, 2004.
[https://doi.org/10.1016/S0047-259X\(03\)00096-4](https://doi.org/10.1016/S0047-259X(03)00096-4)
- [23] HeyGen. LiveAvatar FULL Mode Configuration. Developer documentation.
<https://docs.liveavatar.com/docs/full-mode/configuration>